Codex Patch Smart Router

Model Audit Archive, Merge Decision, and Implementation Direction

Generated: 2026-07-10 • Project repo: codex-patch-smart-router • Local path: /home/l/codex-patch-smart-router

This document archives the model audit material supplied during the design sprint for Codex Patch Smart Router and records the merged implementation decision. It is meant to become the first PDF artifact for the project: a readable technical audit trail plus the next build direction for Codex.

1. Current Project State

• We are not planning the portal in this phase.

• AIOA/AOIA-Core production is paused for this workstream.

• The standalone project is Codex Patch Smart Router, repository slug codex-patch-smart-router.

• The local repo exists at /home/l/codex-patch-smart-router.

• Migration from codex-auto-model-router was completed and local tests previously passed: 41 passed.

• The user reports GitHub authentication was repaired and the repository was pushed successfully.

• The next task is not a rewrite. It is a Knowledge Library upgrade on top of the existing V0 router.

2. My Merged Decision

Final decision: implement a deterministic, data-driven Knowledge Library upgrade. The router should not train an LLM and should not call an LLM to classify prompts in V0. It should score prompt weight using static JSON rule files, hard safety overrides, category scores, complexity/evidence/context/action-danger dimensions, and regression evals.

• Primary architecture source: Sonnet. It gives the strongest ordering constraint: hard safety/risk override must run before category classification is trusted.

• Primary coding architecture source: DeepSeek. It gives the best module additions: scorer.py, preprocessor.py, ScoreCard-style output, immutable rules, and eval-driven tests.

• Best structural loader idea: Gemini. Add a controlled Knowledge Library loader, preferably smart_codex/knowledge.py or a strict config.py load_rules() layer.

• Best prompt-weight framing: Meta AI. Prompt weight means operational cost: risk, context, evidence, complexity, action danger, and consequence before Codex runs.

• Best public-facing summary: Grok. Use it to improve README/SECURITY/PRIVACY and community clarity, not as the implementation source.

3. Non-Negotiable Safety Policy

• dry-run is default for every task

• --execute is always required for real Codex launch

• router.py must never emit an execute-now decision

• launcher.py is the final execution gate

• no raw prompt logging, only prompt_hash and structured routing metadata

• no shell=True and no shlex.split(prompt); prompt must remain one argv element

• no danger-full-access in V0, and config load must reject it

• approval_policy remains on-request in V0

• no SDK/app-server integration in V0

• no invented Codex tools and no invented model names

• model placeholders remain public until codex debug models is used locally

• malformed Knowledge Library JSON must produce CONFIG_ERROR and no execution, not a silent fallback that launches Codex

4. Implementation Decision

Implement the upgrade in this order: safety layer first, scoring second, profile policy third, eval last. Do not start by tuning categories. First protect the execution boundary and hard overrides.

• 1. Add rules/prompt_weight_dimensions.json.

• 2. Add rules/risk_triggers.json and update risk.py so hard overrides short-circuit before category routing.

• 3. Add tests/test_hard_overrides.py.

• 4. Add rules/category_weights.json and upgrade classifier.py to weighted scoring.

• 5. Add rules/complexity_rules.json, evidence_rules.json, context_rules.json, action_danger_rules.json.

• 6. Add smart_codex/scorer.py and optionally smart_codex/preprocessor.py or smart_codex/knowledge.py.

• 7. Update router.py to combine ScoreCard, tie-breakers, risk/action danger, low-confidence fallback.

• 8. Add rules/profile_policy.json and validate official Codex values only.

• 9. Update profiles.py to reject danger-full-access and invalid verbosity/reasoning values.

• 10. Keep launcher.py as final --execute gate; add regression tests.

• 11. Update logging_safe.py to log score dimensions without prompt text.

• 12. Add rules/eval_set_002.jsonl with 80+ cases, using Sonnet cases as the base.

• 13. Run all old tests; old 41 must pass unmodified. Then run new tests.

5. Merge Table

Source	Use in final plan	Reject / fix
Sonnet	Main safety/order architecture; risk-first short-circuit; action_danger separate; launcher owns --execute; strong eval plan.	Remove any auto whitelist from V0; fast profile should be read-only by default.
DeepSeek	scorer.py, preprocessor.py, ScoreCard, immutable JSON rules, regex compile/cache, test_scorer/test_eval_set.	Do not move rules under smart_codex/rules; keep root rules/. No approval_policy=never. No fallback launch on config error.
Gemini	Static Knowledge Library, strict separation rules-vs-code, knowledge.py/config loader, no raw logs, danger-full-access injection test.	Config failure must be CONFIG_ERROR/no execution. Math/literary defaults should be read-only. Deploy/force push must be security/read-only in V0.
Meta AI	Prompt weight as operational cost; 14 dimensions; top-3/mixed scoring; broad safety trigger list.	Remove null sections; confirm_required is not a Codex approval_policy; system_admin/deploy/release force security/read-only.
Grok	Concise architecture framing, README-friendly summary, failure modes, eval_set_002 as CI blocker.	Too short for implementation; do not use as sole Codex prompt.
Kimi upload	Prompt source archive only.	Uploaded file is the audit prompt, not a Kimi answer.

6. Final Build Prompt for Codex - Draft

```
Implement the Knowledge Library upgrade in /home/l/codex-patch-smart-router.
Do not restructure the existing V0 pipeline; extend it with data-driven scoring.
Preserve all existing 41 V0 tests unmodified.

Add root-level rules files:
- rules/prompt_weight_dimensions.json
- rules/category_weights.json
- rules/risk_triggers.json
- rules/complexity_rules.json
- rules/action_danger_rules.json
- rules/evidence_rules.json
- rules/context_rules.json
- rules/profile_policy.json
- rules/tie_breakers.json
- rules/eval_set_002.jsonl

Add or update modules:
- smart_codex/config.py or smart_codex/knowledge.py: load and validate all rules; CONFIG ERROR means no execution.
- smart_codex/risk.py: hard safety override first, before classifier output is trusted.
- smart_codex/classifier.py: weighted category scoring from category_weights.json.
- smart_codex/scorer.py: compute ScoreCard dimensions.
- smart_codex/router.py: combine hard overrides, category score, action danger, evidence/context, tie-breakers, low-
confidence fallback.
- smart_codex/profiles.py: validate profile_policy.json and reject danger-full-access.
- smart_codex/launcher.py: preserve --execute as the only execution gate; never use shell=True; never split prompt
text.
- smart_codex/logging_safe.py: log prompt hash + structured score fields, never raw prompt.
- smart_codex/model_audit.py: keep model placeholders unless codex debug models provides local IDs.

Safety rules:
- high/critical risk -> security -> read-only -> on-request.
- force push, production deploy, release publishing, pipe-to-shell, rm -rf, .env, tokens, API keys -> hard override.
- no auto-execution in V0.
- no danger-full-access.
- no SDK/app-server.
- no invented tools/model names.

Tests to add:
- test_hard_overrides.py
- test_action_danger.py
- test_no_danger_full_access.py
- test_execute_gate.py
- test_no_raw_prompt_logging.py
- test_confidence_fallback.py
- test_tie_breakers.py
- test_eval_set_002.py

Implementation order:
1. risk_triggers + risk.py + hard override tests
2. category_weights + classifier.py
3. scorer dimensions + router integration
4. profile_policy + profiles validation
5. launcher/logging regressions
6. eval_set_002 and pass-rate tuning by editing JSON weights only
7. docs update

Report back with changed files, test counts, failures, and next step.
```

Appendix A - First Short Summary Response

MODEL: unidentified / early small audit response. USE: weak summary, not a design base.

```
To jest szczegółowy prompt audytowo-projektowy (druga iteracja) do budowy narzędzia codex-auto-model-router.
Krótko i po polsku:
To jest specyfikacja techniczna dla inteligentnego wrappera/routera do OpenAI Codex CLI. Celem jest stworzenie
bezpiecznego, lokalnego narzędzia smart-codex, które przed uruchomieniem komendy Codex:
- analizuje prompt użytkownika,
- klasyfikuje zadanie (coding, security, research, math itd.),
- ocenia ryzyko i złożoność,
- wybiera odpowiedni profil/model/ustawienia Codexa,
- wyjaśnia decyzję,
- loguje w bezpieczny sposób (bez przechowywania pełnego promptu),
- uruchamia Codex przez oficjalne mechanizmy (codex --profile ...).
Dlaczego to powstało?
Autor (Łukasz Żuchowski) chce mieć kontrolowany, bezpieczny interfejs do Codexa zamiast uruchamiać go na
ślepo. Szczególnie ważne jest bezpieczeństwo (sandbox, approval policy, read-only dla ryzykownych operacji) i
prywatność.
Kluczowe elementy projektu (V0):
- Python CLI wrapper (smart-codex "twój prompt")
- Deterministyczna klasyfikacja + routing
- Poprawne profile TOML zgodne z aktualnym Codexem
- Bezpieczne logowanie (hash promptu zamiast pełnego tekstu)
- Audyt modeli (codex debug models)
- Testy, dokumentacja, struktura repo gotowa do GitHuba
Podsumowując: To nie jest kod, tylko bardzo dokładna inżynierska specyfikacja (druga passa audytu), na
podstawie której inny model ma teraz zbudować działający projekt.
```

Appendix B - Sonnet Final Architecture Audit

MODEL: Sonnet. ROLE: Architecture Lead. USE: primary source for final architecture.

```
# Final Audit for Codex Patch Smart Router

AUDIT ROLE applied: Architecture Lead (no role was selected in the prompt; Architecture Lead is the correct
default because this audit gates the code design that every other role would review afterward - security,
eval, and maintainer concerns are still addressed in full in Sections 7, 11, and 12 below).

## 1. Verdict

Direction is correct. V0's pipeline (classifier -> risk -> router -> profiles -> launcher -> logging safe) is
the right shape and should not be restructured - only extended with a data layer. Three things must change
before coding starts:

1. Risk must be evaluated before category classification is trusted, not after. Currently the pipeline reads
left-to-right as if classification happens first; the knowledge-library upgrade must make risk.py's hard-
override check a short-circuit that runs first and can bypass classifier.py's output entirely. This is an
ordering constraint, not just a scoring detail - get it wrong and a well-crafted harmless-sounding prompt with
a secret buried in it could route through classifier.py's normal scoring path.
2. Action danger needs its own scored dimension, separate from risk level. V0's risk.py currently conflates is
this prompt about something risky with does this action, if executed, do something dangerous. A prompt like
explain what rm -rf does is security audit-adjacent but action danger = read only analysis; a prompt like run
rm -rf on the build dir is the same category but action danger = destructive operation. These must not
collapse to the same profile decision.
3. --execute gating must be enforced at the launcher boundary, not the router boundary. The router should
never emit an execute now signal; it should only ever emit a routing decision (profile/sandbox/approval).
Enforcement that a real launch requires --execute belongs in launcher.py as a hard gate, so that no future
change to router.py's scoring logic can accidentally cause auto-execution.

Everything else in V0 (dry-run default, prompt hash-only logging, --profile/--model/--sandbox/--cd/--config as
the only Codex CLI surface used) is sound and unchanged by this upgrade.

## 2. Architecture Upgrade

Knowledge Library = pure data layer behind config.py, consumed by classifier.py, risk.py, and router.py. No
module hardcodes keyword lists or thresholds in Python - every number and string that affects a routing
decision lives in rules/. Existing tests should remain stable because they test pipeline plumbing.

## 3. Files To Add

rules/prompt weight dimensions.json - canonical dimension list and scales.
rules/category weights.json - keyword weights per category.
rules/risk triggers.json - hard override groups.
rules/complexity rules.json - complexity signals.
rules/action danger rules.json - action danger mapping.
rules/evidence rules.json - evidence requirement signals.
rules/context rules.json - context requirement signals.
rules/profile policy.json - profile configuration policy.
rules/tie breakers.json - ordered precedence rules.
rules/eval set 002.jsonl - 80+ eval cases.
tests/test eval set 002.py - runs full eval set.
tests/test hard overrides.py - verifies short-circuit before classifier.
tests/test_action_danger.py - verifies explain rm-rf vs run rm-rf distinction.

## 4. Files To Modify

config.py: load rules() and validate all rules; reject danger-full-access.
classifier.py: weighted scoring from category weights.json; pure function.
risk.py: check hard override(prompt) -> Optional[OverrideResult], called before classifier is trusted.
router.py: combine action danger, evidence, context, tie-breakers; never emit execute=true.
profiles.py: load profile policy.json and validate Codex values.
launcher.py: keep --execute gate; add regression guard that scoring cannot loosen it.
logging safe.py: log prompt hash + structured fields only.
model audit.py: warn if placeholders remain; do not invent names.
cli.py: preserve explicit --execute requirement.

## 5. JSON Schemas and Examples

prompt weight dimensions.json defines task category, risk level, complexity level, execution scope,
repo impact, security sensitivity, destructiveness, evidence requirement, context requirement,
math reasoning requirement, creative requirement, research requirement, approval requirement,
confidence_level, action_danger.

category weights.json example:
{
"security audit": {"strong": {"secret": 10, "api key": 10, "token": 10, "private key": 10, "auth": 8,
"oauth": 8, "sandbox": 8, "malware": 10, "exploit": 10, "ssh key": 10, "password": 9}, "weak": {"permissions":
5, "login": 4, "access": 4, "credential": 6}},
"secret handling": {"strong": {"rotate key": 10, "generate token": 10, "store secret": 10, "add .env": 9},
"weak": {"credential value": 6}}
}

risk triggers.json schema includes group, terms, risk, profile, sandbox, approval, execute by default,
confirmation_required, reason.

profile policy.json example:
{
"security": {"task categories": ["security audit", "secret handling", "system admin", "incident response",
"financial_admin"], "sandbox_mode": "read-only", "model_reasoning_effort": "high", "model_verbosity": "high",
```

```
"approval_policy": "on-request", "model": "REPLACE WITH SECURITY MODEL FROM CODEX DEBUG MODELS"}
}
## 6. Category Taxonomy
All twelve suggested additional categories are confirmed: debugging, testing, dependency_management,
release_management, documentation, data_analysis, legal_admin, financial_admin, system_admin,
incident_response, secret_handling, prompt_engineering. They have distinct routing consequences and should not
be collapsed.
## 7. Safety Overrides
Every group has execute_by_default=false. Secrets/keys, auth/identity, malware/exploit, destructive_fs_ops,
elevated_privilege, production_changes, GitHub/cloud_creds, database_ops, payment/billing, legal/PII, force
push, pipe-to-shell.
## 8. Scoring Algorithm
1. Hard override check short-circuits everything.
2. Category scoring only reached if no override fired.
3. Compute confidence from normalized top-two score delta.
4. Apply tie-breakers in fixed order.
5. Compute complexity, evidence, context, action_danger independently.
6. Unknown/low confidence with risk -> security; without risk -> standard with warning.
7. Return routing decision only; no execute flag.
## 9. Profile Routing Policy
fast: simple text/docs overlap, read-only preferred in our merge.
standard: normal coding/debugging/testing.
deep: complex coding/architecture/prompt_engineering.
math: math_theory, read-only unless explicit file write.
security: security_audit, secret_handling, system_admin, incident_response, financial_admin, hard_overrides.
literary: literary/email, read-only.
research: research, grant_work, legal_admin, data_analysis.
repo: repo_operations, release_management, dependency_management, except destructive git ops escalate.
## 10. Eval Set Design
Full eval set 002 should include 80+ cases: 24 categories x at least 2, 12 hard override groups, 9 tie-breaker
rules, ambiguous/filler cases. Representative cases include: meeting_email, simple_text, literary,
normal_coding, complex_coding, architecture, math_theory, security_audit, repo_operations, research,
grant_work, unknown, debugging, testing, dependency_management, release_management, documentation,
data_analysis, legal_admin, financial_admin, system_admin, incident_response, secret_handling,
prompt_engineering, mixed_security/coding, force push, rm -rf, curl|bash.
## 11. Test Plan
test_eval_set_002.py, test_hard_overrides.py, test_action_danger.py, test_no_danger_full_access.py,
test_no_raw_prompt_logging.py, test_execute_gate.py, test_confidence_fallback.py, test_tie_breakers.py.
Existing 41 V0 tests must pass unmodified.
## 12. Security Review
Dry-run remains default. --execute explicit. No raw prompt logs. No shell=True. No danger-full-access. on-
request default approval. No invented tool/model names. No SDK/app-server. No global Codex config mutation.
Hard override before classifier is the highest-value security property.
## 13. Implementation Order
1. prompt_weight_dimensions
2. risk_triggers + risk.py + hard_override_tests
3. category_weights + classifier.py
4. complexity/evidence/context/action_danger + router_scoring_functions
5. tie_breakers + router_apply_tie_breakers
6. profile_policy + profiles_validation
7. config_startup_validation
8. launcher_assertion + execute_gate_test
9. logging_field-set_update
10. model_audit_placeholder_check
11. eval_set_002 + eval_tests
12. action_danger/confidence/tie-breaker_tests
13. full_local_test_run
## 14. Risks / Failure Modes
Override bypass via score drowning; category/action-danger conflation; silent danger-full-access
reintroduction; log leakage via warning/reason fields; eval-set overfitting; tie-breaker order drift;
confidence threshold miscalibration.
## 15. Final Build Prompt For Codex
Implement the Knowledge Library upgrade in /home/l/codex-patch-smart-router following Section 13 exactly.
Create all rules/*.json and .jsonl files, update
config/risk/classifier/router/profiles/launcher/logging/model_audit, add tests, preserve all 41 pre-existing
tests, do not introduce shell=True, danger-full-access, invented tools, invented model names, or raw prompt
logging. Iterate only on rules weights until eval passes.
```

Appendix C - DeepSeek Architecture Lead Audit

MODEL: DeepSeek. ROLE: Architecture Lead. USE: scorer.py/ScoreCard and implementation mechanics.

```
# Final Audit for Codex Patch Smart Router
AUDIT_ROLE: Architecture Lead
## 1. Verdict
The current direction is correct. V0 pipeline, privacy-first logging, and safe defaults form a solid
foundation. The proposed upgrade to a Knowledge Library with deterministic scoring and tie-breaking is the
right path to add intelligence without an LLM.
Before coding, lock down: immutable JSON schemas, purely functional scoring, no external dependencies, no
runtime modification of rules, placeholder model names remain until smart-codex audit-models populates local
IDs.
## 2. Architecture Upgrade
User Prompt -> CLI -> Pre-processor -> Knowledge Library -> Scoring Engine -> Router + Overrides -> Profile
Loader -> Launcher -> Safe Logger.
Knowledge Library location should remain root rules/ in our merge, although DeepSeek originally suggested
smart_codex/rules/. Rules are read at startup, never mutated.
## 3. Files To Add
smart codex/scorer.py - compute scores for all dimensions.
smart codex/preprocessor.py - normalize prompt and flag injection markers.
rules/prompt weight dimensions.json, category weights.json, risk triggers.json, complexity rules.json,
action danger rules.json, evidence rules.json, context rules.json, profile policy.json, tie breakers.json,
eval set 002.jsonl.
tests/test_scorer.py and tests/test_eval_set.py.
## 4. Files To Modify
cli.py: add optional threshold and call new pipeline.
router.py: integrate scoring engine and safety overrides.
classifier.py: replace single-pattern classification with multidimensional scoring.
risk.py: add low/medium/high/critical and action danger.
launcher.py: keep simple, preserve dry-run.
logging safe.py: log additional scoring fields.
config.py: load all rule files.
## 5. JSON Schemas and Examples
prompt weight dimensions.json: dimensions with risk, complexity, evidence requirement, context requirement,
math reasoning, creative requirement, research requirement, action danger, repo impact, security sensitivity,
destructiveness, execution scope, confidence.
category weights.json: one object per category with base risk, complexity, evidence, context, math, creative,
research, action danger, repo impact, security, destructiveness, scope.
risk triggers.json: regex patterns like api key/token/secret/password/credentials and rm
-rf/sudo/chmod/chown/delete all/wipe.
complexity rules.json: multi-file, refactor, architecture, migration, database.
action danger rules.json: force push, git push --force, DROP TABLE, rm -rf /.
evidence rules.json: prove, proof, equation, lemma, citation.
context rules.json: in the context of, given the repo, current project, this codebase.
profile policy.json: decision table mapping risk/action danger/category to profile.
tie breakers.json: hard safety override, security beats coding, destructive beats convenience, math beats
text, grant beats simple text, architecture beats coding, repo beats writing,
unknown_with_risk_signal_to_security, unknown_to_standard.
## 6. Category Taxonomy
Confirms all added categories: debugging, testing, dependency management, release management, documentation,
data analysis, legal admin, financial admin, system admin, incident response, secret handling,
prompt_engineering. Each has profile/risk/complexity and keywords.
## 7. Safety Overrides
Hard coded and/or rules backed: secrets/tokens/passwords, .env/private keys, malware/exploit/CVE, sandbox
escape/permissions, destructive commands, production/deploy/release, cloud credentials/billing, force push/git
destructive, personal data/PII, pipe-to-shell. All execute_by_default=false.
## 8. Scoring Algorithm
Score categories with strong/weak/negative keywords, pick primary with tie-breakers, adjust risk from
triggers, compute complexity/evidence/context/action danger/repo impact/security/destructiveness/execution
scope/confidence, return ScoreCard.
## 9. Profile Routing Policy
DeepSeek proposed fast/standard/deep/math/security/literary/research/repo. In our merge: all approval policy
on-request, no approval=never; math default read-only; safety overrides high/critical -> security.
## 10. Eval Set Design
Provides 40 representative examples and coverage plan for remaining 40. In our merge, Sonnet eval cases are
the base and DeepSeek examples supplement.
## 11. Test Plan
T01-T40: category routing, secrets critical, dry-run, no raw logging, model audit parser, invalid config
rejection, eval set pass rate.
## 12. Security Review
Never use shell=True. Never select danger-full-access. No raw prompt logging. on-request for high risk. No
network telemetry. No .env/auth reads. --execute required. Model placeholders stay until audited.
```

```
## 13. Implementation Order
Create JSONs, add scorer.py, preprocessor.py, classifier wrapper, risk wrapper, router integration, logging
fields, cli, tests, eval set, run old tests, tune JSON weights, docs update.
## 14. Risks / Failure Modes
False positives from security words, conflicting categories, regex performance, malformed JSON, unsafe
override, log growth.
## 15. Final Build Prompt
Implement the rule-based scoring upgrade in /home/l/codex-patch-smart-router with rules files, scorer.py,
preprocessor.py, classifier/risk/router/profiles/logging/cli updates, 80+ eval cases, no safety regression,
old 41 tests pass.
```

Appendix D - Gemini Architecture Lead Audit

MODEL: Gemini. ROLE: Architecture Lead. USE: static Knowledge Library and config-loader separation.

```
# Final Audit for Codex Patch Smart Router
AUDIT_ROLE: Architecture Lead
## 1. Verdict
The current architectural direction is robust and correct. Upgrading V0 to a deterministic, data-driven
Knowledge Library cleanly decouples the routing logic from LLM hallucination risks. The pipeline effectively
uses static analysis (keyword mapping, regex, structural heuristics) to determine prompt weight, risk, and
category before invoking any model execution.
Before coding, strict separation of concerns must be enforced: the Knowledge Library (rules/*.json) must
remain entirely static and independent of Python execution logic. The router consumes these JSON files at
runtime. Gemini suggested fallback to security/read-only on parser failure; in our merge, malformed JSON
becomes CONFIG_ERROR and no execution.
## 2. Architecture Upgrade
User CLI Input -> cli.py -> classifier.py with Knowledge Lib -> risk.py -> router.py -> profiles.py ->
launcher.py -> logging_safe.py -> Codex execution.
## 3. Files To Add
rules/prompt weight dimensions.json, category weights.json, risk triggers.json, complexity rules.json,
action danger rules.json, evidence rules.json, context rules.json, profile policy.json, tie breakers.json,
eval_set_002.jsonl.
## 4. Files To Modify
classifier.py loads category weights; risk.py loads risk/action/complexity; router.py loads tie breakers and
dimensions; profiles.py reads profile_policy; config.py adds rule paths.
## 5. JSON Schemas and Examples
category weights.json examples include math theory and grant work; risk triggers include rm -rf/chmod/git push
--force/password/secret/api key/oauth; profile policy includes security and math profiles. In our merge, math
sandbox is read-only, not workspace-write.
## 6. Category Taxonomy
Gemini approves additions: debugging, dependency management, release management, documentation,
incident response, secret handling. It lists base categories with default profiles and keywords. In our merge,
simple text should map to fast, documentation to fast unless literary style, system admin/security sensitive
to security.
## 7. Safety Overrides
Secrets/keys -> critical/security/read-only/on-request. Destructive -> high/security/read-only. Permissions ->
high/security/read-only. Malware/exploit -> critical/security/read-only. Publishing/deploy -> Gemini proposed
repo/read-only; our merge uses security/read-only in V0. Execution bypass -> critical/security/read-only.
## 8. Scoring Algorithm
Baseline scores, hard override check, keyword scoring, top category, secondary dimensions, tie-breakers, final
mapping.
## 9. Profile Routing Policy
Gemini lists fast/standard/deep/math/security/literary/research/repo. In our merge: all on-request, no danger-
full-access, math/literary read-only defaults, high/critical security.
## 10. Eval Set Design
Gemini gives 10 examples and instructs Codex to expand to 80+. In our merge, this is not sufficient alone;
Sonnet full coverage plan is preferred.
## 11. Test Plan
hard override secrets, destructive commands escalation, math tie-breaker, default dry-run, prompt hash
logging, unknown fallback, eval_set_002 compliance.
## 12. Security Review
No shell=True, prompt hash only, no danger-full-access, no auto-execution, no key touching.
## 13. Implementation Order
Scaffolding, data loader, classifier upgrade, risk override, router synthesis, profile binding, testing/evals.
## 14. Risks / Failure Modes
Token/keyword overlap, JSON exceptions, danger-full-access injection.
## 15. Final Build Prompt
Create rules directory, JSON/JSONL files, modify classifier/risk/router/profiles, enforce tie-breakers and
safety overrides, add tests, preserve safety.
```

Appendix E - Meta AI Prompt Weight Audit

MODEL: Meta AI. ROLE: Prompt Weight / Risk Scoring Auditor. USE: dimensions and trigger taxonomy; nulls removed in merge.

```
Prompt Weight Knowledge Library for Codex Patch Smart Router
Version 0.1 - Audit & Implementation Specification
Repository: codex-patch-smart-router
Status: V0 - Deterministic, No LLM Training
## 1. Executive Summary
Prompt weight is a composite signal that measures how demanding, risky, complex, expensive, context-heavy,
evidence-heavy, or consequential the next Codex task is. It is operational cost, not semantic similarity.
The router must answer before launching Codex: What kind of task is this? How risky is execution? How much
context is needed? How much evidence is needed? How dangerous are implied actions?
V0 principles: no LLM training, deterministic JSON rules, every rule has eval case, hard safety overrides
normal routing, default dry-run, read-only or workspace-write only, no danger-full-access, no raw prompt logs,
no shell=True.
## 2. Prompt Weight Dimensions
Meta defines 14 dimensions and says risk level, security sensitivity, and destructiveness dominate;
complexity/context tune reasoning effort; evidence tunes verbosity and search suggestion.
## 3. Prompt Category Taxonomy
Final V0 taxonomy: 24 categories, base 12 plus debugging, testing, dependency management, release management,
documentation, data analysis, legal admin, financial admin, system admin, incident response, secret handling,
prompt_engineering.
## 4. Weighted Scoring System
Deterministic, auditable, no embeddings. Keep top 3 category scores. Mixed prompts within 15% and second
category security-related should apply tie-breaker. Example: fix frontend bug and check exposed API keys ->
security_audit with hard override.
## 5. Hard Safety Override Library
Triggers include secrets, tokens, API keys, .env files, auth/OAuth, SSH keys, passwords, malware/exploit,
sandbox escape, permissions, delete/remove/wipe, rm -rf, sudo, chmod/chown, production deploy, release
publishing, GitHub token, cloud credentials, database migration, payment/billing, legal documents, personal
data/PII. Some rows used confirm required as approval; in our merge, confirm required is a router
warning/state, not Codex approval_policy. Codex approval_policy remains on-request.
## 6. Model/Profile Routing Policy
fast, standard, deep, math, security, literary, research, repo. All V0 approval policy on-request, no danger-
full-access, dry-run enforced.
## 7. Evidence Requirement Scoring
none, light, source required, multi source, official source only. Official source only includes latest,
official docs, OpenAI, CVE, grant eligibility, legal, compliance.
## 8. Context Requirement Scoring
context small, context medium, context large, context unknown. Large context should force inspect-first and
high effort. Unknown adds confidence penalty and warning.
## 9. Action Danger Scoring
read only analysis, write local files, run tests, git operations, network access, dependency install,
database operation, deployment operation, destructive operation, secret touching operation. V0 does not auto-
execute destructive/deployment/secret/database actions.
## 10. Confidence and Tie-Breaking
Low confidence below 0.45 falls back to standard if no risk, security if any risk. Tie rules: security beats
coding, destructive beats convenience, math beats text when math signals exist, grant beats simple text,
architecture beats normal coding, secret handling beats repo ops when secrets are present, release management
beats docs when publish/deploy/tag present, unresolved tie chooses higher-risk profile.
## 11. Eval Dataset Design
Meta intended eval set 002 with 80+ cases but left null placeholders. In our merge, Sonnet/DeepSeek examples
replace nulls.
## 12. JSON Knowledge Library Schema
Meta lists files and purposes but leaves several schema blocks null. In our merge, nulls are not acceptable;
all JSON files must be complete before Codex implementation.
## 13. Implementation Instructions
Update config.py, classifier.py, risk.py, router.py, profiles.py, launcher.py/cli.py,
tests/test_eval_set_002.py. Preserve existing tests, no raw prompt logs, no shell=True, no danger-full-access,
dry-run default.
## 14. PDF/Documentation Structure
Title, Purpose, Architecture, Knowledge Library, Prompt Weight Dimensions, Routing Rules, Safety Rules, Eval
Dataset, Implementation Instructions, Open Questions, Appendices.
## 15. Open Questions
codex debug models output, available model IDs, profile acceptance, test results, GitHub remote, placeholders
in public repo, threshold tuning, multilingual prompts, logging hash verification.
## 16. Final Build Prompt
Implement Prompt Weight Knowledge Library in /home/l/codex-patch-smart-router, create rules files, update
modules, add tests, preserve safety.
```

Appendix F - Grok Concise Architecture Review

MODEL: Grok. ROLE: general architecture reviewer. USE: concise public framing and failure modes.

```
# Final Audit for Codex Patch Smart Router
## 1. Verdict
Kierunek jest poprawny i gotowy do implementacji. Główne zmiany przed kodowaniem: Knowledge Library jako
czyste JSON, klasyfikator regułami i scoringami, 100% V0 bezpieczeństwa, eval_set_002 jako bloker CI.
## 2. Architecture Upgrade
Prompt -> classifier.py + rules -> risk.py + action danger -> router.py scoring/tie-breakers -> profiles.py +
profile_policy -> launcher.py -> logging_safe.py.
## 3. Files To Add
prompt weight dimensions.json, category weights.json, risk triggers.json, complexity rules.json,
action danger rules.json, evidence rules.json, context rules.json, profile policy.json, tie breakers.json,
eval_set_002.jsonl.
## 4. Files To Modify
classifier.py, risk.py, router.py, profiles.py, config.py, model_audit.py, tests.
## 5. JSON Schemas and Examples
Grok gives concise examples for prompt_weight_dimensions, risk_triggers, profile_policy.
## 6. Category Taxonomy
Short table for security audit, normal coding, math theory, repo operations, research and notes for remaining
categories.
## 7. Safety Overrides
Secrets/keys/tokens -> critical/security/read-only. Destructive -> high/security/read-only. Production
deploy/release -> high/repo/read-only in Grok; our merge makes this security/read-only. Malware/exploit ->
critical/security/read-only.
## 8. Scoring Algorithm
Hard override first, category scoring, risk_score, complexity_score, tie-breakers, map to profile.
## 9. Profile Routing Policy
fast, standard, deep, math, security, repo, research, literary. All on-request and dry-run default.
## 10. Eval Set Design
Minimum 80 lines, examples and coverage plan.
## 11. Test Plan
classifier regressions, safety overrides, eval set, dry-run default, no raw logging.
## 12. Security Review
No raw prompt logging, --execute required, no danger-full-access, no shell=True, no global config mutation,
model placeholders.
## 13. Implementation Order
Rules files, classifier/risk, scoring/tie-breakers, profiles, eval set, tests, README/SECURITY.
## 14. Risks / Failure Modes
Over-triggering security, weak mixed prompt detection, V0 regression, prompt injection in rules mitigated by
static JSON.
## 15. Final Build Prompt
Implement upgrade according to Sections 2-13, preserve V0, add rules, tests, and run local tests before
commit.
```

Appendix G - Kimi Source File: Audit Prompt, Not Answer

SOURCE: uploaded text file. NOTE: this is the prompt sent to models, not a Kimi audit response.

```
FINAL MODEL AUDIT PROMPT — Codex Patch Smart Router Implementation Architecture
AUDIT ROLE: "<INSERT ONE: Architecture Lead | Security Red Team | Eval/Test Engineer | Open Source
Maintainer>"
Project:
Codex Patch Smart Router
Repository slug:
codex-patch-smart-router
Local repo path:
/home/l/codex-patch-smart-router
Public purpose:
A safe local smart router for OpenAI Codex CLI that detects prompt type, task weight, risk, complexity,
context requirement, evidence requirement, and action danger, then selects the correct Codex
model/profile/sandbox/settings before launching a Codex task.
Current confirmed state:
- Repo exists locally at /home/l/codex-patch-smart-router
- Repo has been migrated from old name codex-auto-model-router
- V0 implementation exists
- Previous local tests passed: 41 passed
- GitHub push has been completed by the user
- We are no longer planning the portal
- We are not producing AIOA/AOIA-Core right now
- This is a separate standalone public router project
Important:
Do not summarize this prompt.
Do not repeat this prompt.
Do not ask what to do next.
Do not offer generic help.
Return concrete architecture, code design, JSON content, test plan, and implementation instructions.
Your answer must be usable as direct input for Codex implementation.
---
1. Existing V0 Behavior To Preserve
The current router already has the basic pipeline:
prompt
-> classifier
-> risk estimator
-> router
-> profile selector
-> launcher
-> safe logger
Existing modules:
smart_codex/
cli.py
classifier.py
risk.py
router.py
profiles.py
launcher.py
logging_safe.py
model_audit.py
config.py
Existing rule/config areas:
rules/
profiles/
tests/
examples/
Do not break existing V0 behavior.
Must preserve:
- dry-run default
- --execute required for real Codex launch
- no raw prompt logging
- prompt hash logging only
- no shell=True
- no danger-full-access in V0
- no automatic global Codex config mutation
- no SDK/app-server integration in V0
- no invented Codex tools
- no invented model names
```

```
---
2. Official Codex Constraints
Use only official-style Codex CLI mechanisms:
codex --profile <profile>
codex --model <model>
codex --sandbox <sandbox>
codex --cd <directory>
codex --config key=value
Valid "sandbox_mode" values:
read-only
workspace-write
danger-full-access
V0 must never select:
danger-full-access
Valid "model_verbosity" values:
low
medium
high
Valid "model_reasoning_effort" values:
minimal
low
medium
high
xhigh
Default V0 approval policy:
on-request
Do not use:
sandbox mode = "standard"
sandbox mode = "isolated"
model verbosity = "minimal"
model verbosity = "standard"
model verbosity = "detailed"
model verbosity = "verbose"
model_reasoning_effort = "none"
Do not invent tool names.
Do not add:
tools = ["code-interpreter"]
tools = ["file-search"]
tools = ["git-kit"]
Model placeholders must stay placeholders until local "codex debug models" output is used:
REPLACE WITH FAST MODEL FROM CODEX DEBUG MODELS
REPLACE WITH STANDARD MODEL FROM CODEX DEBUG MODELS
REPLACE WITH DEEP MODEL FROM CODEX DEBUG MODELS
REPLACE_WITH_SECURITY_MODEL_FROM_CODEX_DEBUG_MODELS
---
3. Target Improvement
We are upgrading V0 into a smarter deterministic router.
We are not training an LLM.
We are adding a small Knowledge Library that gives the router a concept of:
prompt weight
Definition:
Prompt weight = how demanding, risky, complex, expensive, context-heavy, evidence-heavy, or consequential the
next Codex task is before Codex runs it.
The smart router should score:
task category
risk level
complexity level
execution scope
repo impact
security sensitivity
destructiveness
evidence requirement
context requirement
math reasoning requirement
creative requirement
research requirement
approval requirement
confidence level
action_danger
Risk scale should become:
low
medium
high
```

critical

Routing rule:

high or critical -> security profile -> read-only -> on-request

V0 still requires:

--execute

No auto-execution is allowed, even for low-risk tasks.

---

4. Required Router Profiles

The router must support these profiles:

fast
standard
deep
math
security
literary
research
repo

Expected rough mapping:

email/simple text/documentation -> fast or literary
normal coding/debugging/testing -> standard
complex coding/architecture/prompt engineering -> deep
math/proof/equation/neutrino/LSC -> math
security/secrets/auth/sandbox/malware/system admin/incident response -> security
research/grants/legal admin/data analysis -> research
repo/git/release/dependency management -> repo
unknown -> standard with warning unless risk signal exists

Hard safety overrides beat normal category scoring.

Example:

fix frontend bug and check exposed API keys

Expected:

category = security audit or secret handling
risk = high/critical
profile = security
sandbox = read-only
approval_policy = on-request

---

5. Required Knowledge Library Files

Design or improve these files:

rules/prompt_weight_dimensions.json
rules/category_weights.json
rules/risk_triggers.json
rules/complexity_rules.json
rules/action_danger_rules.json
rules/evidence_rules.json
rules/context_rules.json
rules/profile_policy.json
rules/tie_breakers.json
rules/eval_set_002.jsonl

Your output must include concrete schemas and meaningful example content.

Do not just say “create these files.”

Give actual JSON examples and implementation-ready structure.

---

6. Required Categories

Base categories:

email
simple text
literary
normal coding
complex coding
architecture
math theory
security audit
repo operations
research
grant work
unknown

Recommended additional categories:

debugging
testing
dependency management
release management
documentation
data analysis
legal_admin

```
financial admin
system admin
incident response
secret handling
prompt_engineering
Your task:
- confirm or reject each added category,
- explain why,
- define default profile/risk/complexity,
- provide strong keywords,
- weak keywords,
- negative keywords,
- example prompts.
---
7. Required Safety Override System
Design hard safety override triggers for:
secrets
tokens
API keys
private keys
.env files
auth
OAuth
SSH keys
passwords
malware
exploit
sandbox escape
permissions
delete/remove/wipe
rm -rf
sudo
chmod
chown
production deploy
release publishing
GitHub token
cloud credentials
database migration
payment/billing
legal documents
personal data / PII
force push
pipe-to-shell
curl | bash
For each group define:
risk level
selected profile
sandbox
approval policy
execute by default
confirmation required
reason
V0 rule:
execute_by_default must always be false
---
8. Required Scoring Design
Design scoring for:
category score
risk score
complexity score
evidence score
context score
action danger score
confidence_score
The design must support mixed prompts.
Required tie breakers:
1. hard safety override beats everything
2. security beats coding
3. destructive beats convenience
4. math theory beats normal text when proof/equation/tensor/neutrino/LSC signals exist
5. grant work beats simple text when funding/proposal/budget signals exist
6. architecture beats normal coding when system/module/refactor signals dominate
7. repo/release operations beat writing tasks when commit/push/tag/release signals exist
8. unknown routes standard with warning unless risk signals exist
9. any risk signal in unknown prompt routes security/read-only
```

```
---
9. Eval Dataset Requirement
Design "rules/eval_set_002.jsonl".
Minimum:
80 examples
Each line format:
{"prompt":"...", "expected_category":"...", "expected_profile":"...", "expected_risk":"...",
"expected_complexity":"...", "reason":"..."}
Must cover:
all base categories
all added categories
mixed prompts
ambiguous prompts
security overrides
repo operations
release operations
dependency management
grant work
math/LSC/neutrino work
frontend/backend bugs
architecture/refactors
harmless writing
shell injection-like prompts
destructive prompts
secret handling
system admin
incident response
prompt engineering tasks
You do not need to list all 80 in full if space is limited, but you must provide:
- at least 30 high-value examples,
- category coverage plan,
- exact instruction that Codex must create the full 80+ case file.
If you can fit all 80+, provide all 80+.
---
10. Required Output Format
Return exactly this structure:
# Final Audit for Codex Patch Smart Router
## 1. Verdict
State whether the current direction is correct and what must change before coding.
## 2. Architecture Upgrade
Give the target architecture diagram and explain where the Knowledge Library fits.
## 3. Files To Add
List each new file and its purpose.
## 4. Files To Modify
List each existing module and exact change required.
## 5. JSON Schemas and Examples
Provide schemas and examples for each rules/*.json file.
## 6. Category Taxonomy
Provide table: category, default profile, risk, complexity, strong keywords, weak keywords, negative keywords.
## 7. Safety Overrides
Provide hard safety trigger table.
## 8. Scoring Algorithm
Give pseudocode for the scoring pipeline.
## 9. Profile Routing Policy
Define each profile and its constraints.
## 10. Eval Set Design
Provide eval examples and test strategy.
## 11. Test Plan
List exact tests to add and expected behavior.
## 12. Security Review
List what must never change.
## 13. Implementation Order
Give exact order for Codex to implement.
## 14. Risks / Failure Modes
List likely implementation bugs and how to catch them.
## 15. Final Build Prompt For Codex
Give a concise prompt Codex can execute to implement this upgrade.
---
11. Role-Specific Focus
If "AUDIT_ROLE = Architecture Lead", focus on:
- module design,
- clean boundaries,
- data-driven rules,
```

```
- how to keep code simple.
If "AUDIT_ROLE = Security Red Team", focus on:
- bypasses,
- prompt injection,
- secrets,
- unsafe routing,
- logging leaks,
- dangerous overrides,
- shell safety.
If "AUDIT_ROLE = Eval/Test Engineer", focus on:
- eval set 002,
- false positives,
- false negatives,
- regression tests,
- scoring thresholds,
- test fixtures.
If "AUDIT_ROLE = Open Source Maintainer", focus on:
- public repo clarity,
- README/SECURITY/PRIVACY,
- package naming,
- examples,
- contributor safety,
- model placeholders,
- GitHub readiness.
---
12. Non-Negotiable Safety Requirements
Preserve:
dry-run default
--execute required
no raw prompt logging
prompt hash only
no shell=True
no danger-full-access
on-request approval policy
no invented tools
no invented model names
no SDK/app-server in V0
no global Codex config mutation
no touching secrets
---
13. Final Instruction
Execute this audit.
Do not summarize the prompt.
Do not ask follow-up questions.
Do not offer general help.
Return implementation-ready architecture, JSON, code design, tests, and final build prompt.
```

Appendix H - Final Notes for Next Work Session

• If the user supplies a real Kimi answer later, append it as a new appendix and rerun the merge table.

• The next executable Codex prompt should be generated only after deciding whether to add knowledge.py or keep load_rules() in config.py.

• Recommended final choice: add smart_codex/knowledge.py for loading and validation; keep config.py for paths and constants.

• Recommended next commit name: Add prompt weight knowledge library.

• Recommended branch: feature/prompt-weight-knowledge-library.